ACTIVIDAD INTEGRADORA

Actividad Corte 1 – Unidad 1: Programación Orientada a Objetos (POO)

Sistema de Gestión Académica Universitaria (SIGAU) – Versión 1.0

Estudiantes	**Código**
Astrid Nayibe Infante Palacio	200810
Wilkinshon Exneider Guisao Cañaveral	200808
Fecha	16 de octubre de 2026

1. Análisis del problema

La universidad **Militar Nueva Granada,** inicia el desarrollo del Sistema de Gestión Académica Universitaria (SIGAU). En **este** módulo **se necesita colocar la** información básica de los estudiantes matriculados en un programa académico.

El sistema permite**:** registrar, consultar, buscar y calcular información. En esta parte del proyecto – versión1, se usarán únicamente arreglos estáticos, que hacen parte de los fundamentos de programación (POO).

1.1 Entidad identificada. La entidad principal del problema es el Estudiante, que se describe con cinco datos: código, nombre completo, edad, programa académico y promedio acumulado. Además, se necesita una clase que gestione el conjunto de estudiantes (almacenarlos, consultarlos y calcular estadísticas), por lo que el problema se resuelve con dos clases.

1.2 Requerimientos funcionales.

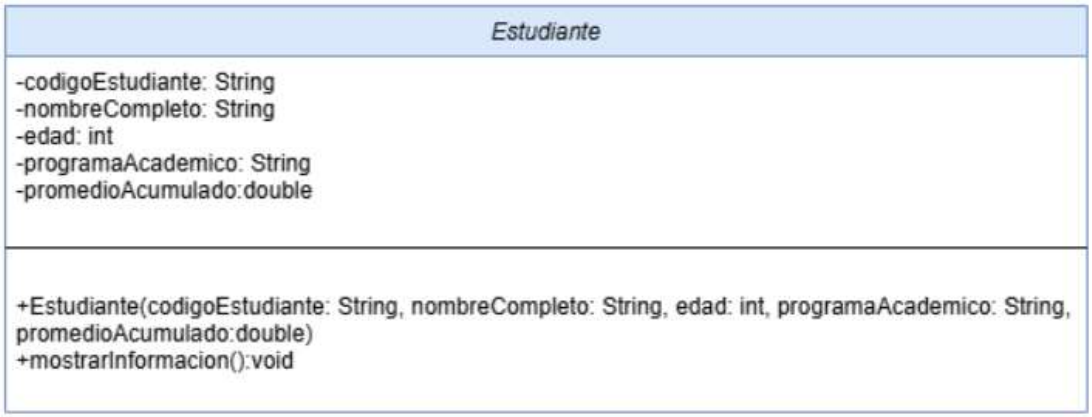
Id	**Requerimiento**	**Entrada**	**Salida**
1	Registrar estudiantes	Código, nombre, edad, programa y promedio	Mensaje de éxito o de error
2	Consultar todos los estudiantes	Ninguna	Listado numerado
3	Buscar estudiante por código	Código	Datos del estudiante o mensaje de no encontrado
4	Calcular el promedio general	Ninguna	Promedio con dos decimales
5	Identificar el mejor promedio	Ninguna	Datos del estudiante con mayor promedio
6	Mostrar número de estudiantes registrados	Ninguna	Cantidad de estudiantes registrados
7	Finalizar la ejecución	Opción 7	Mensaje de despedida

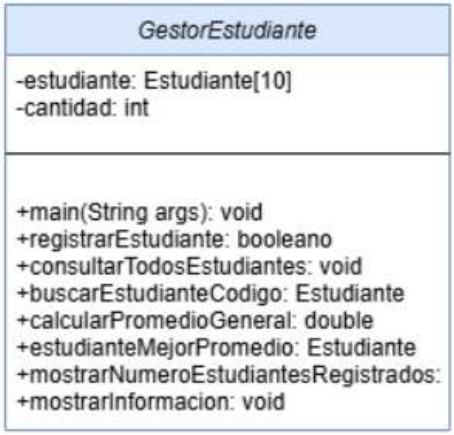
1.3 Información a almacenar (requerimiento de datos). De cada estudiante se guarda la siguiente información, con los tipos de dato usados en la clase Estudiante:

Atributo	Tipo de dato (Java)
Código	String
Nombre completo	String
Edad	int
Programa académico	String
Promedio acumulado	Double

2. Diseño de la solución

La solución usa dos clases dentro del paquete version1. La clase Estudiante modela los datos de una persona; la clase GestorEstudiante contiene el arreglo de objetos Estudiante, el contador de posiciones ocupadas y los métodos que implementan el menú. Entre ambas existe una relación de composición: el gestor contiene un arreglo de estudiantes.





2.1 Diagrama de clases (descripción)

Clase Estudiante	Atributos y métodos
Atributos (privados)	codigo : String nombre : String edad : int programa : String promedio : double
Constructores	Estudiante() Estudiante(codigo, nombre, edad, programa, promedio)
Métodos	getCodigo(), getNombre(), getEdad(), getPrograma(), getPromedio() setCodigo(), setNombre(), setEdad(), setPrograma(), setPromedio() mostrarInformacion() : void

Clase GestorEstudiante	Atributos y métodos
Atributos (privados)	CAPACIDAD : int (constante = 10) estudiante : Estudiante[] cantidad : int scanner : Scanner
Menú y control	main(String[] args) ingresarEstudiantes() mostrarResumen() ejecutarMenu() mostrarMenu()
Funcionalidades	registrarEstudiante() : boolean consultarTodosEstudiantes() : void buscarEstudianteCodigo() : Estudiante calcularPromedioGeneral() : double estudianteMejorPromedio() : Estudiante mostrarNumeroEstudiantesRegistrados() : void mostrarInformacion() : void
Apoyo (privados)	leerEntero() : int leerDecimal() : double

2.2 Estructura de almacenamiento. Los estudiantes se guardan en el arreglo estudiante, de tamaño fijo 10. La variable cantidad indica cuántas posiciones están ocupadas; por eso todos los recorridos van desde la posición 0 hasta cantidad - 1, y el siguiente registro se guarda en la posición cantidad. Cuando cantidad llega a 10, el sistema rechaza nuevos registros.

2.3 Flujo del programa. El método main crea un objeto GestorEstudiante y llama a ingresarEstudiantes(), que pregunta cuántos estudiantes se van a ingresar (entre 1 y el espacio disponible), solicita los datos de cada uno con registrarEstudiante() y al final llama a mostrarResumen(), que imprime todos los estudiantes ingresados con sus promedios, el promedio general, el estudiante con mejor promedio y la cantidad registrada. Luego main llama a ejecutarMenu(). Este método repite, con un ciclo do-while, la lectura de una opción y la ejecución

de la funcionalidad correspondiente mediante una estructura switch, hasta que el usuario elige la opción 7.

3. Descripción de las clases implementadas

El programa está organizado en el paquete version1 y consta de dos clases: Estudiante (entidad del dominio) y GestorEstudiante (clase de control/negocio).

3.1 Clase Estudiante. Permitir crear objetos con su información básica.

- Atributos (privados): codigo (String), nombre (String), edad (int), programa (String), promedio (double).
- Constructores:
- Estudiante() — constructor vacío.
- Estudiante(String codigo, String nombre, int edad, String programa, double promedio) — inicializa los cinco atributos.
- Métodos:
- Getters/setters: getCodigo/setCodigo, getNombre/setNombre, getEdad/setEdad, getPrograma/setPrograma, getPromedio/setPromedio.
- mostrarInformacion() — imprime código, nombre, edad, programa y promedio.
- Encapsulamiento: los atributos son private y se acceden únicamente mediante los métodos públicos get/set.

3.2 Clase GestorEstudiante. Administra el arreglo estático de estudiantes y ofrecer las operaciones del sistema (menú, registro, consulta, búsqueda y cálculos). Contiene el método main.

- Atributos:
- CAPACIDAD — private static final int = 10 (dimensión fija del arreglo).
- estudiante — Estudiante[] (arreglo estático de objetos).
- cantidad — int (número de estudiantes registrados).
- scanner — Scanner (lectura por consola).
- Métodos:
- main(String[] args) — crea el gestor, ejecuta el ingreso inicial, abre el menú y cierra el Scanner.
- ingresarEstudiantes() — pregunta cuántos estudiantes se van a ingresar, los registra uno a uno y muestra un resumen.
- mostrarResumen() — lista los estudiantes y muestra promedio general, mejor promedio y cantidad.
- mostrarMenu() — muestra las 7 opciones.
- ejecutarMenu() — ciclo do-while con switch que conecta cada opción con su método.
- leerEntero() / leerDecimal() — leen y validan la entrada numérica (retornan -1 ante error de formato).
- registrarEstudiante() — valida capacidad, código duplicado, edad y promedio; crea y guarda el objeto. Retorna boolean.
- consultarTodosEstudiantes() — recorre el arreglo e imprime la información de cada estudiante.
- buscarEstudianteCodigo() — búsqueda lineal por código (equalsIgnoreCase). Retorna Estudiante o null.
- calcularPromedioGeneral() — suma los promedios y divide entre la cantidad. Retorna double.
- estudianteMejorPromedio() — recorre el arreglo y retorna el estudiante con mayor promedio.
- mostrarNumeroEstudiantesRegistrados() — imprime la cantidad registrada.
- mostrarInformacion() — muestra el nombre y la versión del sistema (no se usa en el flujo principal).

3.3 Relaciones entre clases y estructura general.

- Agregación: GestorEstudiante "tiene un" arreglo de objetos Estudiante.
- Dependencia: GestorEstudiante usa Scanner para la lectura de datos por consola.
- El sistema se ejecuta desde el main de GestorEstudiante, que coordina el ingreso inicial y el menú interactivo.

4**. Explicación de las principales funcionalidades.** El sistema ofrece un menú de 7 opciones que se ejecuta en un ciclo do-while. Cada opción se conecta, mediante un switch, con un método de la clase GestorEstudiante.

A continuación, se explica el funcionamiento de cada una:

4.1 Registrar estudiante (Opción 1: registrarEstudiante()). Este método agrega un estudiante nuevo al arreglo. Su funcionamiento es el siguiente:
Verifica que haya cupo disponible (cantidad < CAPACIDAD). Si el arreglo está lleno, muestra un mensaje de error y no registra nada.
Solicita el código y comprueba, con equalsIgnoreCase, que ningún otro estudiante lo tenga.
Solicita el nombre completo, la edad, el programa académico y el promedio acumulado.
Valida que la edad sea mayor que 0 y que el promedio esté entre 0.0 y 5.0.
Si los datos son correctos, crea el objeto Estudiante con el constructor parametrizado y lo guarda en estudiante[cantidad]. Aumenta en 1 el contador cantidad y retorna true. Si alguna validación falla, retorna false.

4.2 Consultar todos los estudiantes (Opción 2: consultarTodosEstudiantes()). Este método muestra la lista de estudiantes registrados. Si cantidad es igual a 0, informa que no hay registros. Si hay datos, recorre el arreglo desde el índice 0 hasta cantidad - 1 e invoca mostrarInformacion() de cada objeto, separando un registro de otro con una línea.

4.3 Buscar estudiante por código (Opción 3: buscarEstudianteCodigo()). Este método busca un estudiante a partir de su código. Realiza una búsqueda lineal en la que compara getCodigo() de cada elemento con el código ingresado, usando equalsIgnoreCase(codigoBuscar). Si encuentra coincidencia, muestra la información del estudiante y retorna el objeto. Si no la encuentra, muestra un mensaje y retorna null.

4.4 Calcular el promedio general (Opción 4: calcularPromedioGeneral()). Este método calcula el promedio de todos los estudiantes registrados. Suma el promedio de cada uno y divide el resultado entre cantidad. Antes de dividir, comprueba que cantidad sea mayor que 0 para evitar la división por cero. El valor se muestra con dos decimales (%.2f) y también se retorna.

4.5 Estudiante con mejor promedio (Opción 5: estudianteMejorPromedio()). Este método identifica al estudiante con el promedio más alto. Recorre el arreglo y va almacenando el índice del mayor promedio encontrado hasta ese momento. Al terminar el recorrido, muestra la información de ese estudiante y lo retorna.

4.6 Mostrar la cantidad de estudiantes registrados (Opción 6: mostrarNumeroEstudiantesRegistrados()). Este método muestra cuántos estudiantes hay registrados, valor que corresponde al contador cantidad.

4.7 Finalizar la ejecución (Opción 7: ejecutarMenu()). Con esta opción se termina el programa. Cuando el usuario ingresa 7, el ciclo do-while se detiene, se muestra un mensaje de salida y el main cierra el Scanner.

4.8 Ingreso inicial de estudiantes (ingresarEstudiantes() y mostrarResumen()). Antes de abrir el menú, el programa pregunta cuántos estudiantes se van a ingresar (entre 1 y los espacios disponibles) y los registra de uno en uno. Al finalizar, muestra un resumen con la lista de estudiantes, el promedio general, el estudiante con mejor promedio y la cantidad registrada.

4.9 Menú interactivo (mostrarMenu() y ejecutarMenu())
mostrarMenu() imprime las 7 opciones disponibles.
ejecutarMenu() mantiene el menú activo con un ciclo do-while. Según la opción leída, llama al método correspondiente y, si la opción no es válida, muestra un mensaje de error.

4.10 Lectura y validación de datos (leerEntero() y leerDecimal())
Ambos métodos leen una línea de la consola y la convierten a número dentro de un bloque try-catch. Cuando el usuario ingresa un valor que no es numérico, se captura la excepción

NumberFormatException y el método retorna -1. Con ese valor, las validaciones de la edad, del promedio y de la opción del menú detectan el error y muestran el mensaje respectivo.

5. Pruebas del sistema

Las pruebas se hicieron ingresando al iniciar cinco estudiantes (códigos 202601, 202615, 202620, 202630 y 202635, con los promedios 4.5, 3.8, 4.9, 4.2 y 4.5) y luego registrando estudiantes nuevos hasta llenar el arreglo. La tabla muestra el resultado esperado de cada prueba.

N.º	Prueba	Resultado esperado
0	Al iniciar, ingresar una cantidad inválida (0 o 11) y luego 5	Mensaje de cantidad no válida y, tras ingresar los 5 estudiantes, resumen con la lista completa y promedio general 4.38
1	Opción 2 con 5 estudiantes ingresados	Listado de los estudiantes #1 a #5
2	Opción 3 con el código 202620	Se muestra María Rodríguez (Medicina, 4.9)
3	Opción 3 con el código 999999	Mensaje: no se encontró un estudiante con ese código
4	Opción 4	Promedio general: 4.38
5	Opción 5	Mejor promedio: María Rodríguez, 4.9
6	Opción 6	Cantidad de estudiantes registrados: 5
7	Opción 1 con datos válidos	Estudiante registrado exitosamente; la cantidad sube a 6
8	Opción 1 con un código ya registrado	Error: ya existe un estudiante con ese código
9	Opción 1 con el arreglo lleno (10 estudiantes)	Error: el arreglo está lleno
10	Opción inválida (por ejemplo 9 o una letra)	Mensaje: opción no válida
11	Opción 7	El programa finaliza

5.1 Capturas de la ejecución

Reemplace cada recuadro por la captura de su propia ejecución en Eclipse. Debe ser legible y mostrar el menú junto con el resultado de la opción.

Captura 1. Ingreso de la cantidad de estudiantes, datos de cada uno y promedio general inicial.

```
¿Cuántos estudiantes desea ingresar? (1 a 10): 5

>>> Estudiante 1 de 5

--- Registrar Nuevo Estudiante ---
Código: 202601
Nombre completo: ANDRES RODRIGUEZ
Edad: 20
Programa académico: INGENIERIA INDUSTRIAL
Promedio acumulado (0.0 a 5.0): 4.5
Estudiante registrado exitosamente.

>>> Estudiante 2 de 5

--- Registrar Nuevo Estudiante ---
Código: 202615
Nombre completo: CARLOS LOPEZ
Edad: 22
Programa académico: ADMINISTRACION DE EMPRESAS
Promedio acumulado (0.0 a 5.0): 3.8
Estudiante registrado exitosamente.

>>> Estudiante 3 de 5

--- Registrar Nuevo Estudiante ---
Código: 202620
Nombre completo: MARIA RODRIGUEZ
Edad: 19
Programa académico: MEDICINA
Promedio acumulado (0.0 a 5.0): 4.9
Estudiante registrado exitosamente.

>>> Estudiante 4 de 5

--- Registrar Nuevo Estudiante ---
Código: 202630
Nombre completo: JUAN PEREZ
Edad: 25
Programa académico: DERECHO
Promedio acumulado (0.0 a 5.0): 4.2
Estudiante registrado exitosamente.

>>> Estudiante 5 de 5

--- Registrar Nuevo Estudiante ---
Código: 202635
Nombre completo: FREDA CANTILLO
Edad: 32
Programa académico: INGENIERIA INFORMATICA
Promedio acumulado (0.0 a 5.0): 4.5
```

Captura 2. Menú principal y listado de estudiantes (opción 2).

```
===== SISTEMA DE GESTIÓN ACADÉMICA UNIVERSITARIA =====
1. Registrar estudiante
2. Mostrar todos los estudiantes
3. Buscar estudiante por código
4. Calcular promedio general
5. Mostrar estudiante con mejor promedio
6. Mostrar cantidad de estudiantes registrados
7. Salir
Seleccione una opción: 2

--- Lista de Estudiantes ---

Estudiante #1:
Código: 202601
Nombre: ANDRES RODRIGUEZ
Edad: 20
Programa: INGENIERIA INDUSTRIAL
Promedio: 4.5
------------------------------

Estudiante #2:
Código: 202615
Nombre: CARLOS LOPEZ
Edad: 22
Programa: ADMINISTRACION DE EMPRESAS
Promedio: 3.8
------------------------------

Estudiante #3:
Código: 202620
Nombre: MARIA RODRIGUEZ
Edad: 19
Programa: MEDICINA
Promedio: 4.9

Estudiante #4:
Código: 202630
Nombre: JUAN PEREZ
Edad: 25
Programa: DERECHO
Promedio: 4.2
------------------------------

Estudiante #5:
Código: 202635
Nombre: FREDA CANTILLO
Edad: 32
Programa: INGENIERIA INFORMATICA
Promedio: 4.5
------------------------------
```

Captura 3. Búsqueda por código, con un código existente y uno inexistente (opción 3).

```
===== SISTEMA DE GESTIÓN ACADÉMICA UNIVERSITARIA =====
1. Registrar estudiante
2. Mostrar todos los estudiantes
3. Buscar estudiante por código
4. Calcular promedio general
5. Mostrar estudiante con mejor promedio
6. Mostrar cantidad de estudiantes registrados
7. Salir
Seleccione una opción: 3
Ingrese el código a buscar: 202620

Estudiante encontrado:
Código: 202620
Nombre: MARIA RODRIGUEZ
Edad: 19
Programa: MEDICINA
Promedio: 4.9
```

```
===== SISTEMA DE GESTIÓN ACADÉMICA UNIVERSITARIA =====
1. Registrar estudiante
2. Mostrar todos los estudiantes
3. Buscar estudiante por código
4. Calcular promedio general
5. Mostrar estudiante con mejor promedio
6. Mostrar cantidad de estudiantes registrados
7. Salir
Seleccione una opción: 3
Ingrese el código a buscar: 202699
No se encontró un estudiante con el código: 202699
```

Captura 4. Promedio general y estudiante con mejor promedio (opciones 4 y 5).

```
===== SISTEMA DE GESTIÓN ACADÉMICA UNIVERSITARIA =====
1. Registrar estudiante
2. Mostrar todos los estudiantes
3. Buscar estudiante por código
4. Calcular promedio general
5. Mostrar estudiante con mejor promedio
6. Mostrar cantidad de estudiantes registrados
7. Salir
Seleccione una opción: 4
El promedio general de los estudiantes es: 4,38
```

```
===== SISTEMA DE GESTIÓN ACADÉMICA UNIVERSITARIA =====
1. Registrar estudiante
2. Mostrar todos los estudiantes
3. Buscar estudiante por código
4. Calcular promedio general
5. Mostrar estudiante con mejor promedio
6. Mostrar cantidad de estudiantes registrados
7. Salir
Seleccione una opción: 5

Estudiante con mejor promedio:
Código: 202620
Nombre: MARIA RODRIGUEZ
Edad: 19
Programa: MEDICINA
Promedio: 4.9
```

Captura 5. Cantidad de estudiantes registrados (opción 6).

```
===== SISTEMA DE GESTIÓN ACADÉMICA UNIVERSITARIA =====
1. Registrar estudiante
2. Mostrar todos los estudiantes
3. Buscar estudiante por código
4. Calcular promedio general
5. Mostrar estudiante con mejor promedio
6. Mostrar cantidad de estudiantes registrados
7. Salir
Seleccione una opción: 6
Cantidad de estudiantes registrados: 5
```

Captura 6. Registro exitoso de un estudiante (opción 1).

```
===== SISTEMA DE GESTIÓN ACADÉMICA UNIVERSITARIA =====
1. Registrar estudiante
2. Mostrar todos los estudiantes
3. Buscar estudiante por código
4. Calcular promedio general
5. Mostrar estudiante con mejor promedio
6. Mostrar cantidad de estudiantes registrados
7. Salir
Seleccione una opción: 1

--- Registrar Nuevo Estudiante ---
Código: 202642
Nombre completo: TOMAS GONZALEZ
Edad: 35
Programa académico: INGENIERIA INFORMATICA
Promedio acumulado (0.0 a 5.0): 4.3
Estudiante registrado exitosamente.
```

Captura 7. Validación de código repetido y de arreglo lleno.

```
===== SISTEMA DE GESTIÓN ACADÉMICA UNIVERSITARIA =====
1. Registrar estudiante
2. Mostrar todos los estudiantes
3. Buscar estudiante por código
4. Calcular promedio general
5. Mostrar estudiante con mejor promedio
6. Mostrar cantidad de estudiantes registrados
7. Salir
Seleccione una opción: 1

--- Registrar Nuevo Estudiante ---
Código: 202601
Error: Ya existe un estudiante con el código 202601
```

Captura 8. Opción inválida y salida del sistema (opción 7).

```
===== SISTEMA DE GESTIÓN ACADÉMICA UNIVERSITARIA =====
1. Registrar estudiante
2. Mostrar todos los estudiantes
3. Buscar estudiante por código
4. Calcular promedio general
5. Mostrar estudiante con mejor promedio
6. Mostrar cantidad de estudiantes registrados
7. Salir
Seleccione una opción: 8
Opción no válida. Intente de nuevo.
```

```
===== SISTEMA DE GESTIÓN ACADÉMICA UNIVERSITARIA =====
1. Registrar estudiante
2. Mostrar todos los estudiantes
3. Buscar estudiante por código
4. Calcular promedio general
5. Mostrar estudiante con mejor promedio
6. Mostrar cantidad de estudiantes registrados
7. Salir
Seleccione una opción: 7
Saliendo del sistema...
```

6. Reflexión final.

Con el desarrollo del SIGAU se vio que la Programación Orientada a Objetos sirve mejor que la programación estructurada para representar un problema real, porque el código se organiza en clases que corresponden a cosas concretas, como la clase Estudiante con sus datos y métodos, y así resulta más fácil de leer y de modificar. También se notó que se debe aprender lo básico de Java: variables, operadores, estructuras de control y arreglos. Esto es indispensable para armar el menú y las operaciones de la clase GestorEstudiante: registrar, buscar o calcular promedios.

En el proyecto se utilizaron arreglos estáticos, que tienen un tamaño fijo, que nos permitió practicar los conceptos básicos antes de pasar a estructuras avanzadas.